

Adaptive Tracing in Microservice Systems

Comparative Evaluation of Controller Families, Runtime-Change Behavior, and Design Taxonomy

Dan Bhattarai

May 18, 2026

Abstract

This paper presents a comparative study of adaptive tracing across three microservice applications: Train-Ticket, the Timescale OpenTelemetry Demo, and Spring Petclinic Microservices. The study has three analytical objectives: first, to compare reinforcement-learning and non-reinforcement-learning tracing controllers under matched operating conditions; second, to examine how reinforcement-learning controllers behave under targeted runtime changes such as latency spikes, error bursts, and throughput drops; and third, to synthesize the implemented methods into a practical taxonomy for engineering use. The results show that no controller dominates across all environments. Q-Learning is the strongest overall method on Train-Ticket, the rule-based baseline gives the best final latency on the Timescale application, and Bandit is the strongest Petclinic policy on final runtime metrics. Under runtime-change scenarios, Bandit wins the most latency cases overall, while SARSA wins the most throughput cases. The main engineering conclusion is that controller quality is system-dependent, and adaptive tracing should be selected according to runtime stability, operational transparency requirements, and the quality of the feedback signals available for control.

Contents

1	Introduction	3
2	Study Objectives and Scope	3
3	Experimental Systems	3
4	Methodology	3
4.1	Adaptive tracing controllers	3
4.2	Measured outputs	4
4.3	Study design	4
4.4	Methodological caveats	4
5	Comparative Evaluation of Controller Families	5
6	Controller Behavior Under Runtime Changes	6
7	Taxonomy and Design Guidance	7

8	Discussion	11
8.1	What the project shows overall	11
8.2	Operational guidance	11
8.3	Reward should not dominate the final interpretation	11
9	Threats to Validity and Limitations	11
10	Conclusion	12
A	Study Artifacts	13
B	Additional Readings Consulted	13

1 Introduction

Distributed tracing is a core observability mechanism for microservice applications, but a fixed tracing configuration imposes a persistent tradeoff. Higher sampling improves diagnostic visibility at the cost of additional telemetry volume, while lower sampling reduces overhead but weakens failure analysis and performance diagnosis. Adaptive tracing addresses this tension by modifying the tracing rate at runtime in response to observed system behavior. The study is grounded in established distributed tracing literature and platforms, including Dapper [1], X-Trace [2], Canopy [3], Pivot Tracing [4], OpenTelemetry [5], Jaeger [6], and Zipkin [7], together with project readings on adaptive observability and RL-guided logging [8].

The practical question is not whether adaptation is useful in principle, but which form of adaptation is strongest under realistic operating conditions. This paper therefore treats adaptive tracing as a control problem and evaluates multiple controller families rather than a single adaptive algorithm. The resulting contribution combines a cross-system comparison, a runtime-change study, and a method taxonomy grounded in implemented experiments rather than only conceptual classification.

2 Study Objectives and Scope

The paper is organized around three analytical objectives:

- Objective A:** compare reinforcement-learning and non-reinforcement-learning adaptive tracing controllers under matched system conditions;
- Objective B:** evaluate how reinforcement-learning controllers behave under latency spikes, error bursts, and throughput drops; and
- Objective C:** derive a practical taxonomy of adaptive tracing strategies based on the completed experiments.

The evidence base consists of 30 controller-comparison runs, 2 fixed-rate baselines, and 36 runtime-change reinforcement-learning runs across three applications.

3 Experimental Systems

4 Methodology

4.1 Adaptive tracing controllers

The study evaluates five adaptive controllers. Q-Learning and SARSA are value-learning reinforcement-learning methods. Bandit is a simpler action-value controller with weaker assumptions about longer-term dynamics. Rule is a threshold-based heuristic baseline. K-Means is a clustering-based baseline that maps recent runtime states to representative sampling rates. The common action space is a discrete set of tracing rates: 0.05, 0.10, 0.20, 0.50, and 0.80.

Table 1: Experimental systems used in the study and their role in the evidence base.

System	Domain	Technical role in the study	Observability / tracing context	Comparative winner
Train-Ticket	Railway ticket booking benchmark	Largest and most stable application in the study; strongest evidence base	Python microservices with adaptive tracing controller and experiment runners	Q-Learning
Timescale Open-Telemetry Demo	Password-generation microservice application	Lightweight tracing-first microservice system with strong observability	OpenTelemetry Collector, Promscale, TimescaleDB, Jaeger, Grafana	Rule
Spring Petclinic Microservices	Spring Cloud business-domain microservice application	Most realistic business application, but also the noisiest result set	API gateway, service discovery, config server, Zipkin, Prometheus	Bandit

4.2 Measured outputs

The primary outputs are latency, throughput (QPS), error rate, and trace totals. For reinforcement-learning methods, average reward is also reported. Reward is useful for RL-internal interpretation, but it is not treated as a universally comparable system-quality score because reward design differs across the applications, especially in the Timescale system. Data quality is tracked through decision counts and zero-window counts, because sparse windows weaken the reliability of adaptive-control conclusions.

4.3 Study design

4.4 Methodological caveats

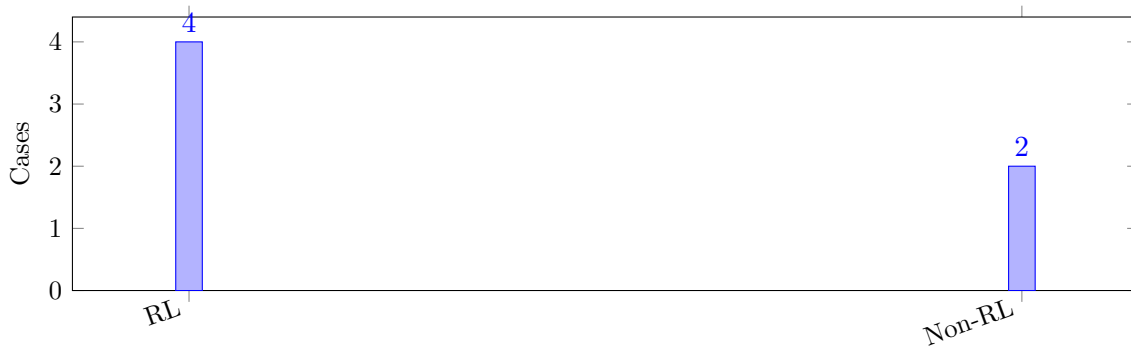
- Reward is reported as contextual reinforcement-learning evidence, not as a uniformly comparable cross-system winner metric.
- The applications are not symmetric in evidence quality. Train-Ticket is the strongest result set, Timescale is valid but has reward-design caveats, and Petclinic is complete but much sparser.
- Fault injection mechanisms differ by application because the systems expose different operational controls. The study therefore emphasizes within-system comparisons first and cross-system synthesis second.

Table 2: Design structure by analytical objective.

Objective	Compared methods	Scenario structure	Main outputs
Comparative evaluation	Q-Learning, SARSA, Bandit, Rule, K-Means	Healthy and faulted per system	Final latency, final QPS, error rate, contextual RL reward
Runtime-change analysis	Q-Learning, SARSA, Bandit	Healthy, latency spike, error burst, throughput drop	Latency/QPS winners per scenario, average selected rate, data quality
Taxonomy synthesis	Taxonomy over implemented methods	Uses completed comparison and runtime-change evidence; no new experiment matrix	Method categories, decision signals, behavior patterns, operational implications

5 Comparative Evaluation of Controller Families

The cross-system comparison shows that reinforcement-learning controllers are consistently competitive, but there is no universal winner across the three applications. Q-Learning is strongest on Train-Ticket, the rule-based baseline is strongest on Timescale final latency, and Bandit is strongest on Petclinic final runtime metrics. The evidence therefore supports a contingent rather than absolute view of controller quality.

**Figure 1:** Latency winner family split across six comparison cases**Table 3:** Cross-system summary of controller-comparison winners.

Application	Scenario	Lowest latency	Value	Highest QPS	Value	Contextual RL reward leader	Avg war
Train-Ticket	Healthy	Bandit	118.58 ms	SARSA	62.551	Q-Learning	0.10
Train-Ticket	Faulted	Q-Learning	96.29 ms	Q-Learning	35.360	Q-Learning	0.07
Timescale Open-Telemetry Demo	Healthy	Rule	133.85 ms	SARSA	2.630	SARSA	0.37

Application	Scenario	Lowest latency	Value	Highest QPS	Value	Contextual RL reward leader	Avg. reward
Timescale Telemetry Demo	Open-Faulted	Rule	211.29 ms	K-Means	2.275	Q-Learning	0.50
Spring Petclinic Microservices	Healthy	Bandit	15.86 ms	Bandit	0.945	Bandit	0.21
Spring Petclinic Microservices	Faulted	Bandit	8.11 ms	Bandit	4.418	Bandit	1.37

Interpretation. Train-Ticket provides the clearest evidence for Q-Learning because it combines strong faulted performance, strong contextual reward, and a mature decision horizon. Timescale shows that a transparent heuristic can still outperform reinforcement learning on final latency, which prevents an overgeneralized conclusion that learning-based control is always superior. Petclinic shows that a simpler reinforcement-learning family member, Bandit, can be preferable when the runtime signal is noisier and operational stability matters more than deeper value learning.

6 Controller Behavior Under Runtime Changes

Controller behavior under runtime change is not uniform. Across the twelve system-scenario cases, Bandit wins the most latency cases (5), SARSA wins the most QPS cases (7), and contextual reward leadership is split across all three reinforcement-learning policies. This indicates that different controllers react differently to degradation patterns rather than sharing a common adaptation profile.

Table 4: Cross-system summary of runtime-change winners.

Application	Scenario	Latency winner	Value	QPS winner	Value	Contextual RL reward leader	Avg. reward
Train-Ticket	Healthy	Q-Learning	54.64 ms	SARSA	68.727	Q-Learning	2.2526
Train-Ticket	Latency Spike	Q-Learning	73.76 ms	SARSA	34.769	Bandit	0.0405
Train-Ticket	Error Burst	Bandit	78.15 ms	Q-Learning	45.003	SARSA	0.0496
Train-Ticket	Throughput Drop	Bandit	78.09 ms	SARSA	24.638	Bandit	0.0283
Timescale Telemetry Demo	Open-Healthy	SARSA	198.71 ms	SARSA	1.931	Q-Learning	0.4598
Timescale Telemetry Demo	Open-Latency Spike	Bandit	344.65 ms	Bandit	1.605	Q-Learning	11.9646
Timescale Telemetry Demo	Open-Error Burst	Q-Learning	334.96 ms	SARSA	1.447	SARSA	0.4667
Timescale Telemetry Demo	Open-Throughput Drop	SARSA	140.85 ms	SARSA	3.330	SARSA	0.3228

Application	Scenario	Latency winner	Value	QPS winner	Value	Contextual RL reward leader	Avg. reward
Spring Petclinic Microservices	Healthy	SARSA	10.05 ms	Bandit	3.037	Bandit	0.8426
Spring Petclinic Microservices	Latency Spike	Bandit	5.23 ms	Q-Learning	7.942	Q-Learning	1.7679
Spring Petclinic Microservices	Error Burst	Q-Learning	12.11 ms	SARSA	4.644	SARSA	0.6268
Spring Petclinic Microservices	Throughput Drop	Bandit	17.81 ms	Q-Learning	2.519	Q-Learning	0.7116

Table 5: System-level data quality across the two experimental blocks.

System	Comparison runs	Comparison avg. decisions	Comparison zero windows	Runtime-ch
Train-Ticket	10	24.3	0	12
Timescale Open-Telemetry Demo	10	4.0	0	12
Spring Petclinic Microservices	10	1.9	2	12

Interpretation. Train-Ticket remains the strongest environment for learning-oriented policies because it supplies long decision horizons and stable traffic. Timescale exposes reward-design weakness most clearly; the latency-spike case shows that contextual reward can appear favorable even when latency becomes extreme, so runtime metrics must remain primary. Petclinic remains usable but sparse. Its results still contribute meaningfully, but the short decision horizon makes the claims there weaker than in Train-Ticket.

7 Taxonomy and Design Guidance

Adaptive tracing is best understood as a family of control strategies rather than one technique. The completed project supports a five-part practical taxonomy: fixed-rate baseline, rule-based control, clustering-based control, bandit-style reinforcement-learning control, and value-learning reinforcement-learning control.

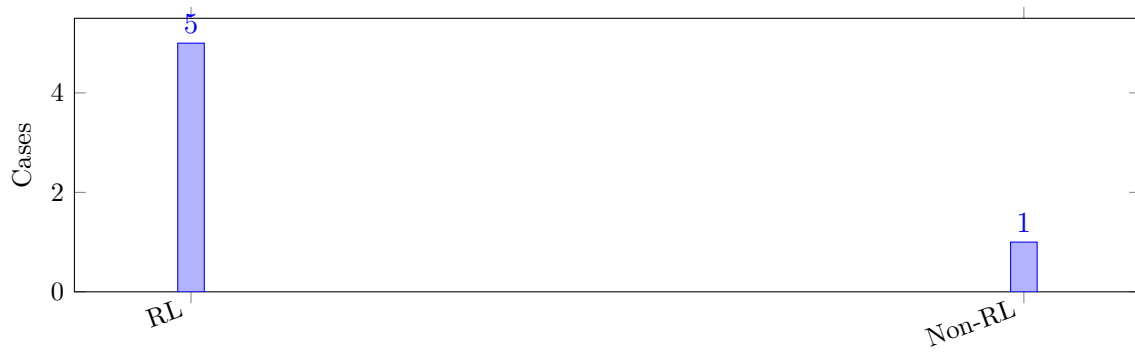


Figure 2: QPS winner family split across six comparison cases

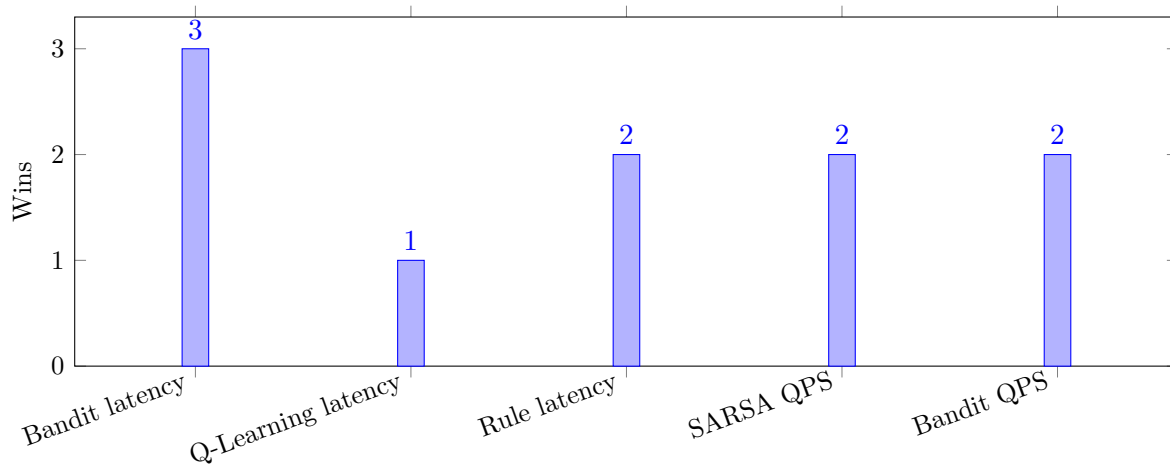


Figure 3: Selected policy win counts in the comparison study

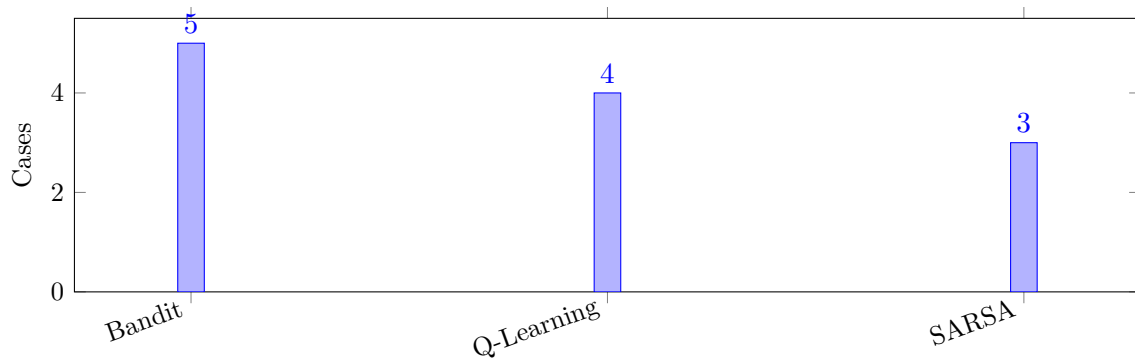


Figure 4: Latency wins under runtime-change scenarios

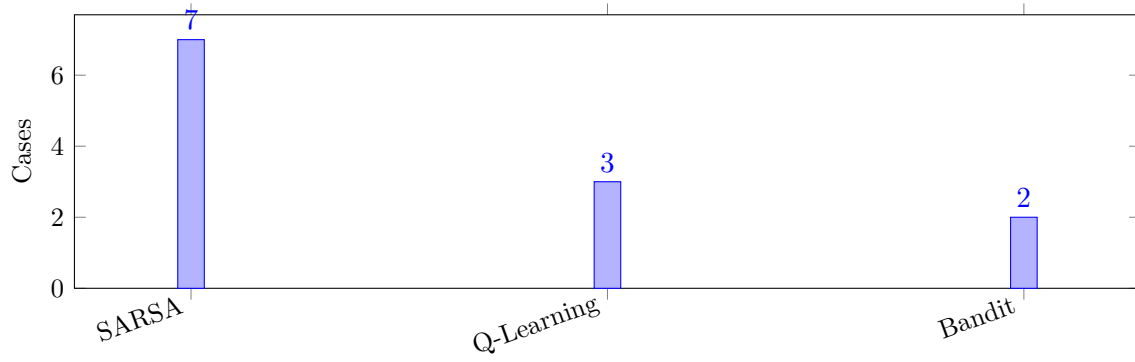


Figure 5: QPS wins under runtime-change scenarios

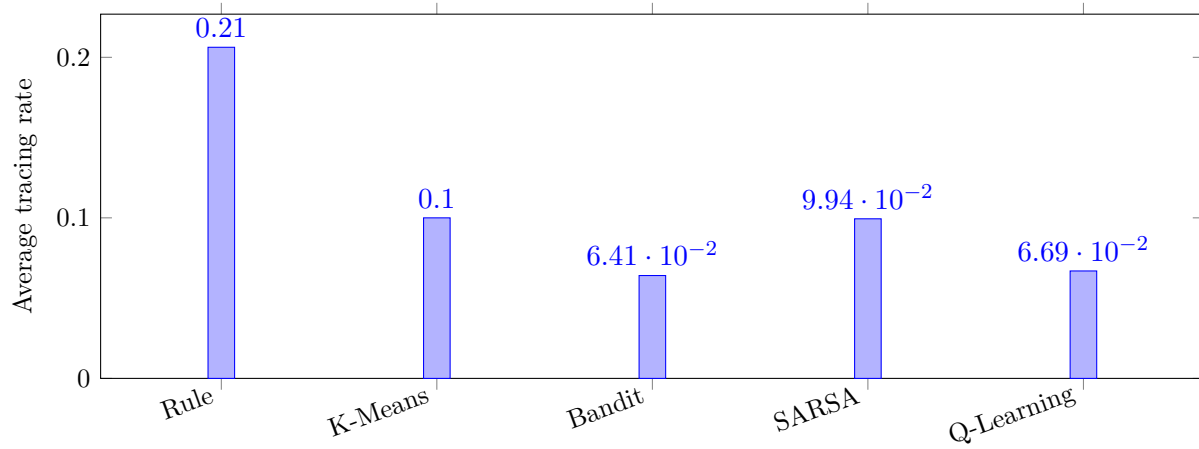


Figure 6: Average selected tracing rate across the full study

Table 6: Practical taxonomy of adaptive tracing strategies.

Method	Type	Main goal	Decision signals	Operational overhead	Observed practical takeaway
Fixed Rate	Static baseline	No runtime adaptation; constant configured rate	Configuration time only	None after startup	Reference point for what is lost without adaptation.
Rule	Heuristic threshold control	Change tracing when thresholds are crossed	Thresholds on latency, errors, counts, or similar runtime signals	Low	Strong when system behavior is predictable and operational transparency matters.
K-Means	Clustering-based control	Map similar runtime states to representative rates	Recent latency, throughput, and error features grouped into clusters	Moderate	Useful middle ground between static thresholds and full reinforcement learning.
Bandit	Reinforcement-learning action-value control	Choose the rate with the best immediate observed payoff	Short-horizon reward from recent behavior	Moderate	Strong on non-stationary systems where low overhead adaptation is valuable.
SARSA	Reinforcement-learning value learning	Learn state-action values on-policy while exploring	State bins derived from runtime metrics plus current action	High	Most exploratory RL method used in this project; useful when control is difficult and exploration helps.
Q-Learning	Reinforcement-learning value learning	Learn long-run state-action value off-policy	State bins derived from runtime metrics and delayed reward	High	Strongest optimization when enough observations and state-action feedback are available.

**Figure 7:** Project-specific conceptual ladder from fixed-rate baselines to value-learning reinforcement-learning controllers. This is not a measured ranking; it summarizes relative modeling and operational complexity.

Why the synthesis matters. The completed experiments do not support a single universal controller. Instead, they support a taxonomy in which controller families differ in goals, decision signals, exploration behavior, and operational cost. This is the appropriate abstraction for making

deployment decisions in practice.

8 Discussion

8.1 What the project shows overall

The strongest project-level conclusion is that controller choice must be matched to the system. Q-Learning is strongest when the environment supplies rich and stable feedback, as in Train-Ticket. Rule-based control remains highly competitive when latency can be improved through transparent threshold logic, as in the Timescale system. Bandit becomes attractive when the system is operationally noisier, as in Petclinic.

8.2 Operational guidance

- Use **Q-Learning** when long decision horizons and stable telemetry make long-run value learning realistic.
- Use **Bandit** when quick, low-overhead adaptation is needed and the environment is noisy or sparse.
- Use **Rule** when transparency, predictability, and ease of explanation are more important than model sophistication.
- Use **K-Means** when a moderate data-driven controller is preferred over manual thresholds.
- Treat **SARSA** as the most exploratory reinforcement-learning option in this project, useful when controlled exploration is acceptable.

8.3 Reward should not dominate the final interpretation

Reward was necessary to drive the reinforcement-learning controllers, but it cannot be allowed to dominate the final report. The Timescale results demonstrate why: the reward signal can look favorable even when the runtime outcome is poor. For that reason, the final synthesis prioritizes latency, QPS, error rate, and data quality over reward in the main findings.

9 Threats to Validity and Limitations

- **Unequal evidence quality:** Train-Ticket is materially stronger than the other two systems. Petclinic is the weakest due to short runs and sparse decision windows.
- **Reward comparability:** reward functions differ across systems, so cross-system reward comparisons are contextual rather than absolute.
- **Scenario heterogeneity:** the exact fault or degradation mechanism differs by application because the systems expose different control surfaces.
- **Different telemetry backends:** the systems use different data paths and observability stacks, which is realistic but introduces methodological heterogeneity.

- **Limited repetition:** the study uses completed matrices rather than repeated statistical replications for every scenario-policy combination.

10 Conclusion

This project evaluated adaptive tracing across three microservice systems through a controller-comparison study, a runtime-change study, and a taxonomy synthesis. The combined evidence shows that reinforcement-learning-based adaptive tracing is consistently competitive but not universally dominant: Q-Learning leads Train-Ticket, Rule leads Timescale final latency, and Bandit leads Petclinic final runtime metrics. The runtime-change analysis shows that controllers respond differently to degradation patterns rather than exhibiting one common adaptation profile. The taxonomy synthesis shows that adaptive tracing is best understood as a family of control strategies, not as a single technique.

The final engineering conclusion is direct: **there is no single best adaptive tracing controller for all microservice systems.** The appropriate choice depends on the stability of the runtime signal, the need for operational transparency, and the quality of the feedback available for learning.

References

- [1] Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspan, S., and Shanbhag, C. *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*. Google technical report, 2010.
- [2] Fonseca, R., Porter, G., Katz, R. H., Shenker, S., and Stoica, I. *X-Trace: A Pervasive Network Tracing Framework*. NSDI, 2007.
- [3] Kaldor, J., Mace, J., Bejda, M., Gao, E., Kuropatwa, W., O'Neill, J., Ong, K., Schaller, B., Shan, P., Viscomi, B., Venkataraman, V., Veeraraghavan, K., Werner, K., and Zhou, Y. *Canopy: An End-to-End Performance Tracing and Analysis System*. SOSP, 2017.
- [4] Mace, J., Roelke, R., and Fonseca, R. *Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems*. SOSP, 2015.
- [5] OpenTelemetry Project. *OpenTelemetry Documentation and Demo Materials*. Online documentation consulted during instrumentation and architecture review. Accessed May 18, 2026.
- [6] Jaeger Project. *Jaeger Documentation*. Online documentation consulted during the tracing-tooling review. Accessed May 18, 2026.
- [7] Zipkin Project. *Zipkin Documentation*. Online documentation consulted for the Petclinic tracing backend. Accessed May 18, 2026.
- [8] *An Adaptive Logging System (ALS): Enhancing Software Logging with Reinforcement Learning Techniques*. Supplemental project reading consulted during the literature review.

A Study Artifacts

The synthesis in this paper was built from the completed experiment outputs and the reviewed cross-system reports. The main local artifacts are:

- `/Users/dan/rq1_all_three_systems_report.html`
- `/Users/dan/rq2_all_three_systems_report.html`
- `/Users/dan/rq3_adaptive_tracing_taxonomy_report.html`
- `/Users/dan/train-ticket-python/experiment_results`
- `/Users/dan/timescale-otel-demo/experiment_results`
- `/Users/dan/spring-petclinic-microservices/experiment_results`

B Additional Readings Consulted

Additional locally stored readings informed the literature review and framing of the study. These included:

- `/Users/dan/Downloads/An Adaptive Logging System (ALS): Enhancing Software Logging with Reinforcement Learning Techniques.pdf`
- `/Users/dan/Downloads/nsdi23-zhang-lei.pdf`
- `/Users/dan/Downloads/review.pdf`
- `/Users/dan/Downloads/https:arxiv.org:pdf:2509.13852.pdf`